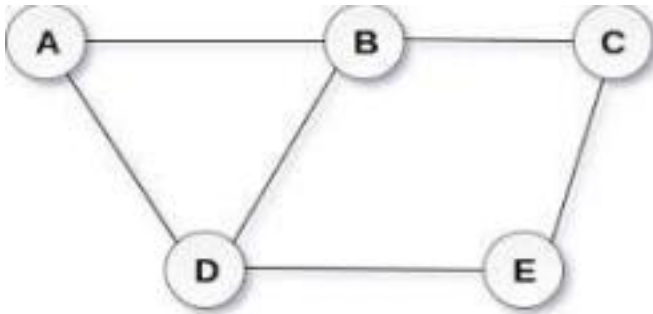


UNIT III – GRAPHS AND NETWORKED SYSTEMS

Graphs – Definitions, Traversals – BFS, DFS, Shortest-Path Algorithms, Dijkstra's Algorithm, Minimum Spanning Tree: Prim's Algorithm, Kruskal's Algorithm, Applications in: Social networking (friend suggestion), GPS and route finding, Dependency resolution (course planning, software builds)

3.1 GRAPHS – DEFINITIONS

A graph G is defined as an ordered set (V, E) , where $V(G)$ represents the set of vertices and $E(G)$ represents the edges that connect these vertices.



Undirected Graph

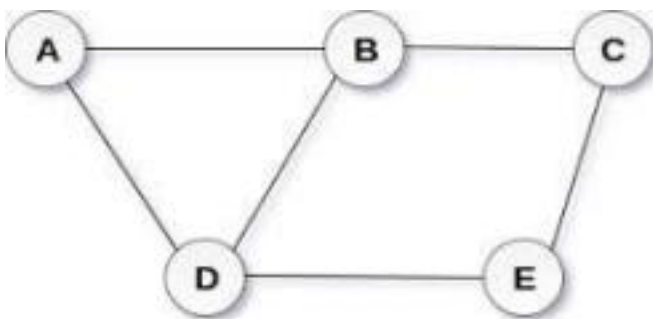
$V(G) = \{A, B, C, D \text{ and } E\}$

$E(G) = \{(A, B), (B, C), (A, D), (B, D), (D, E), (C, E)\}$.

Types of Graphs

DIRECTED AND UNDIRECTED GRAPHS

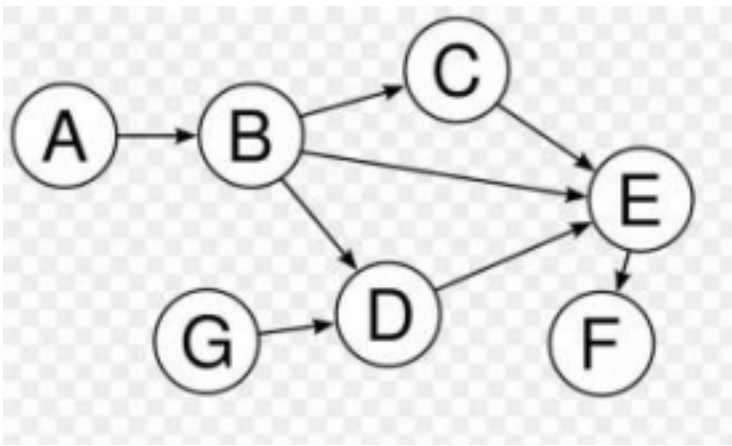
In an undirected graph, edges do not have any direction associated with them.



Undirected Graph

DIRECTED GRAPHS

In a directed graph, edges have direction associated with them. If there is an edge from A to B , then there is a path from A to B but not from B to A .



GRAPH TERMINOLOGY

Adjacent nodes or neighbours

$e = (u, v)$ that connects nodes u and v , the nodes u and v are the end points and are said to be the adjacent nodes or neighbours.

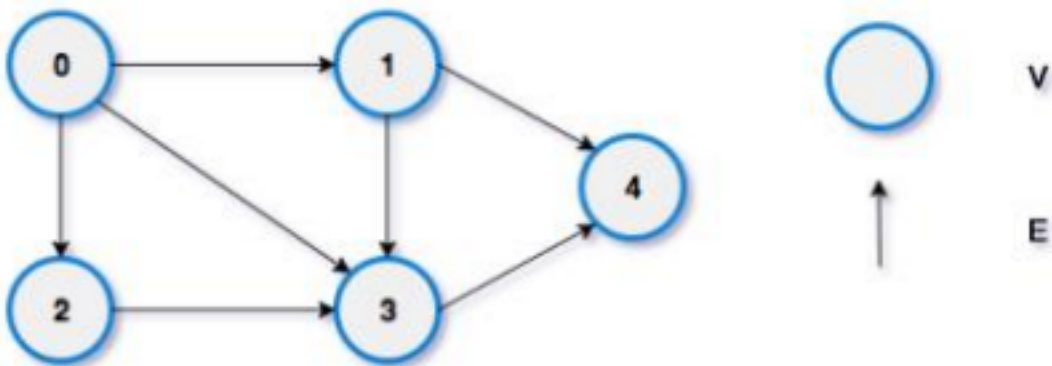
Degree of a node

Total number of edges containing the node u

Cycle

A path in which the first and the last vertices are same.

Degree of a node



In

degree: In-degree of a vertex is the number of edges coming to the vertex

Out-degree- Out-degree of a vertex is the number edges which are coming out from the vertex.

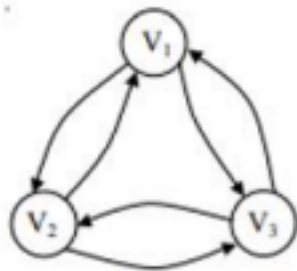
Weighted Graph

A graph is said to be weighted graph if every edge in the graph is assigned a weight or value. It can be directed or undirected graph

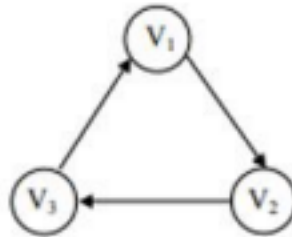


Strongly Connected Graph

If there is a path from every vertex to every other vertex in a directed graph then it is said to be strongly connected graph. Otherwise, it is said to be weakly connected graph.



Strongly Connected Graph



Weakly Connected Graph

Graph Representation

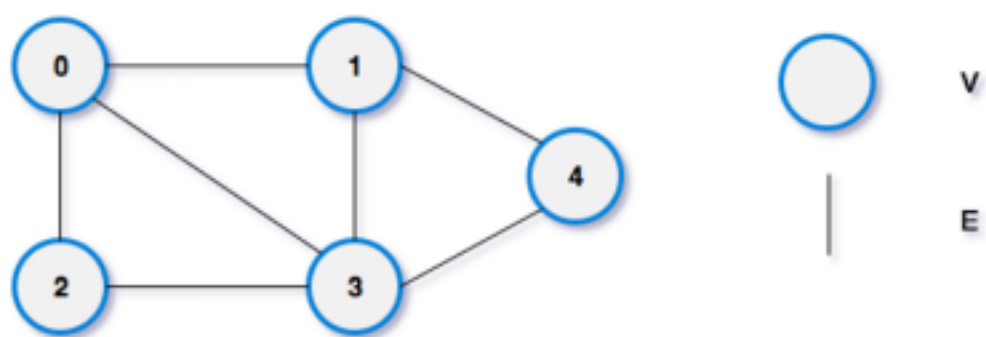
We can easily represent the graphs using the following ways,

1. Adjacency matrix
2. Adjacency list

Adjacency Matrix Representation of Graph

- Adjacency matrix is a sequential representation.
- If a graph has n vertices, we use $n \times n$ matrix to represent the graph. Let's assume the $n \times n$ matrix as $adj[n][n]$.
- if there is an edge from vertex i to j , mark $adj[i][j]$ as 1. i.e. $adj[i][j] == 1$ •
- if there is no edge from vertex i to j , mark $adj[i][j]$ as 0. i.e. $adj[i][j] == 0$

Undirected Graph

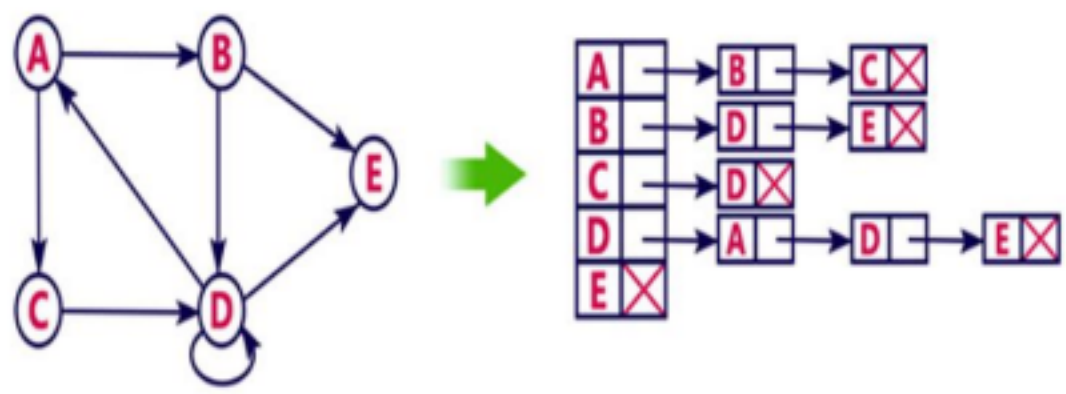


Adjacency Matrix of Undirected Graph

	0	1	2	3	4
0	0	1	1	1	0
1	1	0	0	1	1
2	1	0	0	1	0
3	1	1	1	0	1
4	0	1	0	1	0

Adjacency List Representation of Graph

- Adjacency list is a linked representation.
- In this representation, for each vertex in the graph, we maintain the list of its neighbors. It means, every vertex of the graph contains list of its adjacent vertices.
- We have an array of vertices, which is indexed by the vertex number and for each vertex v, the corresponding array element points to a singly linked list of neighbors of v.



3.2 Traversals – BFS, DFS

Breadth-first traversal

Traversing the graph means examining all the nodes and vertices of the graph.

Two standard methods

- Breadth First Search
- Depth First Search
-

Breadth First Search (BFS) Algorithm

- Breadth first search is a graph traversal algorithm that starts traversing the graph from root node and explores all the neighbouring nodes.
- Then, it selects the nearest node and explore all the unexplored nodes.
- The algorithm follows the same process for each of the nearest node until it finds the goal.

Algorithm

Steps to implement breadth first search

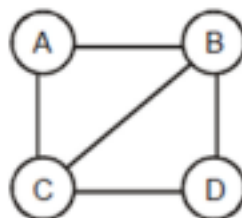
Step 1: Choose any node in the graph, designate it as the search node and mark it as visited.

Step 2: Using the adjacency matrix of the graph, find all the unvisited adjacent nodes to the search node and enqueue them into the queue Q.

Step 3: Then the node is dequeued from the queue. Mark that node as visited and designate it as the new search node.

Step 4: Repeat step 2 and 3 using the new search node. Step 5: This process continues until the queue Q which keeps track of the adjacent nodes is empty.

Example:



Adjacency matrix

	A	B	C	D
A	0	1	1	0
B	1	0	1	1
C	1	1	0	1
D	0	1	1	0

Implementation

1. Let 'A' be the source vertex. Mark it to as visited.
2. Find the adjacent unvisited vertices of 'A' and enqueue then into the queue.
Here B and C are adjacent nodes of A

B	C	
---	---	-------

and B and C are enqueued.

3. Then vertex 'B' is dequeued and its adjacent vertices C and D are taken from the adjacency matrix for enqueueing. Since vertex C is already in the queue, vertex D alone is enqueued.

C	D	
---	---	-------

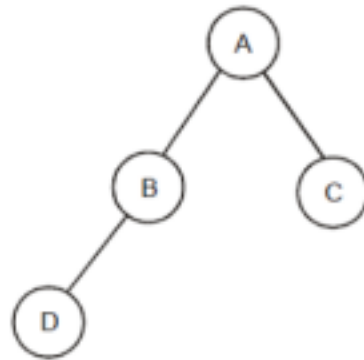
Here B is dequeued, D is enqueued.

4. Then vertex 'C' is dequeued and its adjacent vertices A, B and D are found out. Since vertices A and B are already visited and vertex D is also in the queue, no enqueue operation takes place.

D	
---	-------

Here C is dequeued

5. Then vertex 'D' is dequeued. This process terminates as all the vertices are visited and the queue is also empty.



Breadth first spanning tree

Applications of breadth first search

1. To check whether the graph is connected or not.

DEPTH-FIRST SEARCH ALGORITHM

Depth first Search or Depth first traversal is a recursive algorithm for searching all the vertices of a graph or tree data structure.

- Depth-first search begins at a starting node A which becomes the current node.
- Process a neighbour of A, then a neighbour of neighbour of A, and so on.

During the execution of the algorithm, if we reach a path that has a node N that has already been processed, then we backtrack to the current node. • Otherwise, the unvisited (unprocessed) node becomes the current node. **Algorithm**

To implement the Depth first Search perform the following

Steps :

Step : 1 Choose any node in the graph. Designate it as the search node and mark it as visited.

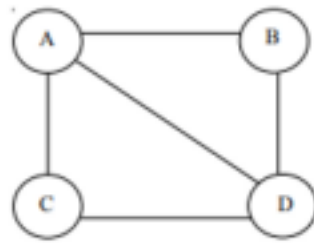
Step : 2 Using the adjacency matrix of the graph, find a node adjacent to the search node that has not been visited yet. Designate this as the new search node and mark it as visited.

Step : 3 Repeat step 2 using the new search node. If no nodes satisfying (2) can be found, return to the previous search node and continue from there.

Step : 4 When a return to the previous search node in (3) is impossible, the search from the originally chosen search node is complete.

Step : 5 If the graph still contains unvisited nodes, choose any node that has not been visited and repeat step (1) through (4).

Example :-

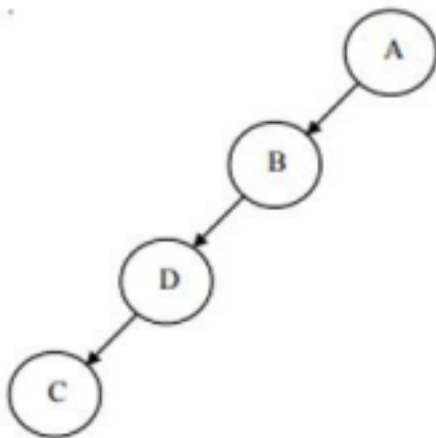


Adjacency Matrix

	A	B	C	D
A	0	1	1	1
B	1	0	0	1
C	1	0	0	1
D	1	1	1	0

Implementation

1. Let 'A' be the source vertex. Mark it to be visited.
2. Find the immediate adjacent unvisited vertex 'B' of 'A' Mark it to be visited.
3. From 'B' the next adjacent vertex is 'd' Mark it has visited.
4. From 'D' the next unvisited vertex is 'C' Mark it to be visited.



Depth First Spanning Tree

Applications of Depth First Search

1. To check whether the undirected graph is connected or not.
2. To check whether the connected undirected graph is Biconnected or not.
3. To check the a Acyclicity of the directed graph

3.3 SHORTEST PATH ALGORITHMS:

An algorithm to find the shortest distance path between the source and destination vertices is called the shortest path algorithm.

Types of shortest path problem

- i. Single source shortest path

Given an input graph $G = (V, E)$ and a distinguished vertex S , find the shortest path from S to every other vertex in G .

Example: Dijkstra's algorithm

(weighted graph and unweighted graph).

ii. All pairs shortest path problem

Given an input graph $G = (V, E)$. Find the shortest path from each vertex to all vertices in a graph.

Dijkstra's algorithm

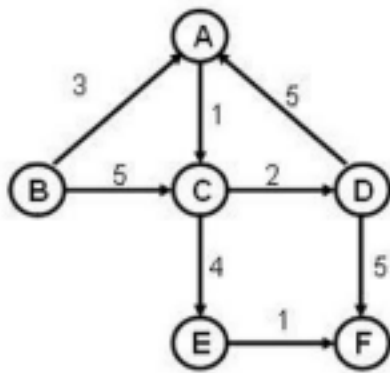
Weighted Graph

The general method to solve the single source shortest path problem is known as Dijkstra's algorithm. It applied to weighted graph.

Procedure

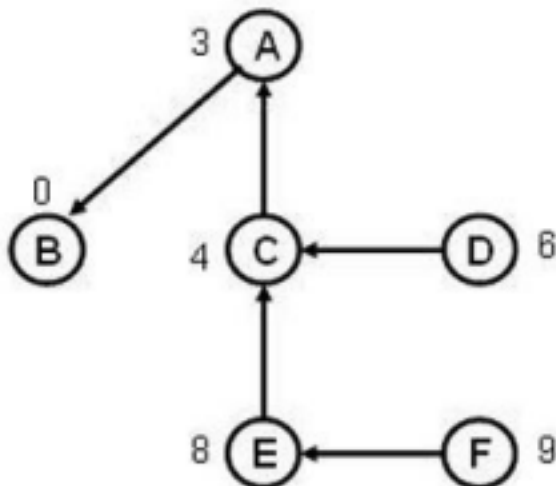
- It uses greedy technique.
- It proceeds in stages.
- It selects a vertex v , which has the smallest d_v among all the unknown vertices and declares the shortest path from s to v is known.
- The remainder consists of updating the value of d_w .
- We should set $d_w = d_v + C_{v, w}$, if the new value for d_w would an improvement.

Example: Find the shortest path for the following graph. Tracing Dijkstra's algorithm starting at vertex B:



Pass:	initially	1	2	3	4	5	6	Shortest distance	Predecessor
Active vertex:		B	A	C	D	E	F		
A	∞	3						3	B
B	0							0	-
C	∞	5	4					4	A
D	∞	∞	∞	6				6	C
E	∞	∞	∞	8	8			8	C
F	∞	∞	∞	∞	11	9		9	E

The resulting vertex-weighted graph is:



Algorithm Analysis

Time complexity of this algorithm $O(|E| + |V|^2) = O(|V|^2)$

3.4 MINIMUM SPANNING TREE: PRIM'S ALGORITHM • A Spanning tree of an undirected graph, G is a tree formed from graph edges that connects all vertices of G.

- A Minimum Spanning tree of an undirected graph, G is a tree formed from graph edges that connects all vertices of G at lowest cost.
- A minimum spanning tree exists if and only if G is connected. The number of edges in the minimum spanning tree is $|V| - 1$.

- The minimum spanning tree is a tree because it is acyclic, it is spanning because it covers every vertex, and it is minimum because it covers with minimum cost.
- The minimum spanning tree can be created using two algorithms, that is prim's algorithm and kruskal's algorithm.

3.5 PRIM'S ALGORITHM

In this method, minimum spanning tree is constructed in successive stages. In each stage, one node is picked as a root and an edge is added and thus an associated vertex is added to the tree.

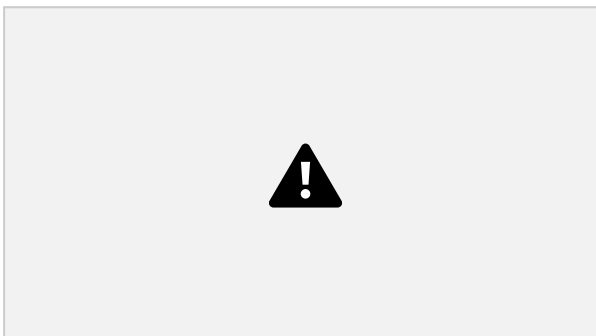
The Strategy

1. One node is picked as a root node (u) from the given connected graph.
2. At each stage choose a new vertex v from u, by considering an edge (u,v) with minimum cost among all edges from u, where u is already in the tree and v is not in the tree.
3. The prim's algorithm table is constructed with three parameters.

They are

- known – known vertex i.e., processed vertex is indicated by 1. Unknown vertex is indicated by zero.
 - d_v - Weight of the shortest edge connecting v to the known vertex.
 - p_v - It contains last vertex to cause a change in d_v .
4. After selecting the vertex v, the update rule is applied for each unknown w adjacent to v. The rule is $d_w = \min(d_w, C_{w,v})$.

Example:



Prim's Algorithm after each stage



Steps

- i. v1 is selected as initial node and construct initial configuration of the table.

v	Known	dv	pv
V1	0	0	0
V2	0	∞	0
V3	0	∞	0
V4	0	∞	0
V5	0	∞	0
V6	0	∞	0
V7	0	∞	0

- ii. v1 is declared as known vertex. Then its adjacent vertices v2, v3, v4 are updated.

$$T[v2].dist = \min(T[v2].dist, C_{v1,v2}) = \min(\infty, 2) = 2$$

$$T[v3].dist = \min(T[v3].dist, C_{v1,v3}) = \min(\infty, 4) = 4$$

$$T[v4].dist = \min(T[v4].dist, C_{v1,v4}) = \min(\infty, 1) = 1$$

v	Known		pv
V1	1	0	0
V2	0	2	V1
V3	0	4	V1

V4	0	1	V1
V5	0	∞	0
V6	0	∞	0
V7	0	∞	0

iii. Among all adjacent vertices V2, V3, V4. V1 -> V4 distance is small. So V4 is selected and declared as known vertex. Its adjacent vertices distance are updated.

- V1 is not examined because it is known vertex.
- No change in V2 , because it has $d_v = 2$ and the edge cost from V4 -> V2 = 3.

$$T[v3].dist = \min(T[v3].dist, C_{v4,v3}) = \min(4, 2) = 2$$

$$T[v5].dist = \min(T[v5].dist, C_{v4,v5}) = \min(\infty, 7) = 7$$

$$T[v6].dist = \min(T[v6].dist, C_{v4,v6}) = \min(\infty, 8) = 8$$

$$T[v7].dist = \min(T[v7].dist, C_{v4,v7}) = \min(\infty, 4) = 4$$

v	Known	dv	pv
V1	1	0	0
V2	0	2	V1
V3	0	2	V4
V4	1	1	V1
V5	0	7	V4
V6	0	8	V4
V7	0	4	V4
V7	0	4	V4

iv. Among all either we can select v2, or v3 whose $d_v = 2$, smallest among v5, v6 and v7.

- v2 is declared as known vertex.
- Its adjacent vertices are v1, v4 and v5. v1, v4 are known vertex, no change in their d_v value.

$$T[v5].dist = \min(T[v5].dist, C_{v2,v5}) = \min(7, 10) = 7$$

v	Known		pv
---	-------	--	----

V1	1	0	0
V2	1	2	V1
V3	0	2	V4
V4	1	1	V1
V5	0	7	V4
V6	0	8	V4
V7	0	4	V4

v. Among all vertices v3’s dv value is lower so v3 is selected. v3’s adjacent vertices are v1, v4 and v6. No changes in v1 and v4.

$T[v6].dist = \min(T[v6].dist, C_{v3,v6}) = \min(8, 5) = 5$

v	Known		pv
V1	1	0	0
V2	1	2	V1
V3	1	2	V4
V4	1	1	V1
V5	0	7	V4
V6	0	5	V3
V7	0	4	V4

vi. Among v5, v6, v7, v7’s dv value is lesser, so v7 is selected. Its adjacent vertices are v4, v4, and v6. No change in v4. $T[v5].dist = \min(T[v5].dist, C_{v7,v5}) = \min(7, 6) = 6$

$T[v6].dist = \min(T[v6].dist, C_{v7,v6}) = \min(5, 1) = 1$

v	Known		pv
V1	1	0	0
V2	1	2	V1
V3	1	2	V4

V4	1	1	V1
V5	0	6	V7
V6	0	1	V7
V7	1	4	V4

vii. Among v5 and v6, v6 is declared as known vertex. v6’s adjacent vertices are v3, v4, and v7, no change in dv value, all are known vertices.

v	Known	dv	pv
V1	1	0	0
V2	1	2	V1
V3	1	2	V4
V4	1	1	V1
V5	0	6	V7
V6	1	1	V7
V7	1	4	V4

viii. Finally v5 is declared as known vertex. Its adjacent vertices are v2, v4, and v7, no change in dv value, all are known vertices.

v	Known	dv	pv
V1	1	0	0
V2	1	2	V1
V3	1	2	V4
V4	1	1	V1
V5	1	6	V7
V6	1	1	V7
V7	1	4	V4

The minimum cost of spanning tree is 16.

Algorithm Analysis

The running time is $O(|V|^2)$ in case of adjacency list and $O(|E| \log |V|)$ in case of binary heap.

3.6 KRUSKAL'S ALGORITHM

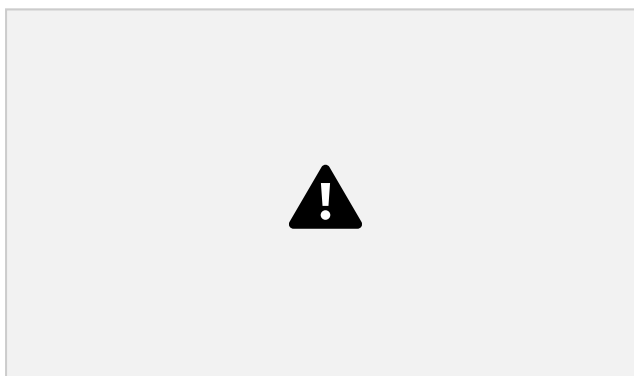
A second greedy strategy is repeatedly to select the edges in order of smallest weight and accept an edge if it does not cause a cycle.

Steps:

1. Initially, there are $|V|$ single-node trees. Adding an edge merges two trees into one.
2. When the algorithm terminates, there is only one tree, and this is the minimum spanning tree.
3. The algorithm terminates when enough edges are accepted.

The strategy

- i. The edges are built into a minheap structure and each vertex is considered as a single node tree.
 - ii. The delete-min operation is used to find the minimum cost edge (u,v) .
 - iii. The vertices u and v are searched in the spanning tree set S and if the returned sets are not same then (u,v) is added to the set s with the constraint that adding (u,v) will not create a cycle in spanning tree set S .
 - iv. Repeat step (ii) and (iii) until a spanning tree is constructed with $|V| - 1$ edges.
- Example**



- i. Initially all the vertices are single node trees.
- ii. Select the smallest edge v_1 to v_4 , both the nodes are different sets, it does not form cycle.
- iii. Select the next smallest edge v_6 to v_7 . These two vertices are different sets; it does not form a cycle, so it is included in the MST.

- iv. Select the next smallest edge v_1 to v_2 . These two vertices are different sets; it does not form a cycle, so it is included in the MST.
- v. Select the next smallest edge v_3 to v_4 . These two vertices are different sets; it does not form a cycle, so it is included in the MST.
- vi. Select the next smallest edge v_2 to v_4 both v_2 and v_4 are same set, it forms cycle so $v_2 - v_4$ edge is rejected.
- vii. Select the next smallest edge v_1 to v_3 , it forms cycle so $v_1 - v_3$ edge is rejected.
- viii. Select the next smallest edge v_4 to v_7 , it does not form a cycle so it is included in the tree.
- ix. Select the next smallest edge v_3 to v_6 , it forms a cycle so $v_3 - v_6$ edge is rejected.
- x. Select the next smallest edge v_5 to v_7 , it does not form a cycle so it is included in the tree.

Edge Weight Action

(v_1, v_4) 1 Accepted

(v_6, v_7) 1 Accepted

(v_1, v_2) 2 Accepted

(v_3, v_4) 2 Accepted

(v_2, v_4) 3 Rejected

(v_1, v_3) 4 Rejected

(v_4, v_7) 4 Accepted

(v_3, v_6) 5 Rejected

(v_5, v_7) 6 Accepted

(v_3, v_6) 5 Rejected

(v_5, v_7) 6 Accepted

Figure: Action of Kruskal's algorithm on G

All the nodes are included. The cost of minimum spanning tree = $16 (2 + 1 + 2 + 4 + 1 + 6)$.



Routine for kruskals algorithm

```
void kruskal( graph G )
{
    int
    EdgesAccepted
    d; DisjSet S;
    PriorityQueue
    H; vertex u, v;
    SetType uset,
    vset; Edge e;
    Initialize( S ); // form a single node tree ReadGraphIntoHeapArray(
    G, H ); BuildHeap( H );
    EdgesAccepted = 0;
    while( EdgesAccepted < NumVertex-1 )
    {
        e = DeleteMin( H ); // Selection of minimum edge uset = Find( u, S
```

```

); vset = Find( v, S );
if( uset != vset ) { /* accept the edge */ EdgesAccepted++; SetUnion( S, uset,
vset
);
}
}
}
}

```

- The appropriate data structure is the union/find algorithm
- The worst-case running time of this algorithm is $O(|E| \log |E|)$, which is dominated by the heap operations. Notice that since $|E| = O(|V|^2)$, this running time is actually $O(|E| \log |V|)$.

3.7 APPLICATIONS IN: SOCIAL NETWORKING (FRIEND SUGGESTION)

Graphs and networked systems are applied to friend suggestions by modeling users as nodes and their relationships as edges in a network.

Algorithms analyze these connections to find users who share common friends or similar interests. By examining two-degree-of-separation connections (friends-of-friends), the system identifies potential friends with shared network patterns and behaviors.

Graph databases efficiently store and query these relationships, enabling complex, multi-hop analysis to recommend new connections based on commonalities and network topology.

Graph Representation: A user's social network is represented as a graph where each user is a node (or vertex), and a friendship or connection is an edge.

Network Analysis: The system analyzes the topology of the network to find users who are connected to the target user's existing friends.

Two-Degree-of-Separation: Potential friend candidates are often found within the "friends-of-friends" (FOF) network, which are users separated by two degrees of connection.

CS3301 DS

Behavioral Similarity: Algorithms then look for patterns in how these FOF users are connected to other nodes, identifying users who exhibit similar behaviors or interests to the root user.

Recommendations: Based on the analysis, users who share similar connection patterns and interests with the root user are recommended as potential new friends.

Graph Databases: Modern social platforms use graph databases to efficiently store and query these user relationships, allowing for complex analysis and real time updates.

Key aspects of graph analysis for friend suggestions

Nodes: Users within the social network.

Edges: Connections (friendships) between users.

Clusters: Groups of users who are densely connected, indicating potential communities or shared interests.

Centrality: Measures the importance of a node within the network, helping to identify influential users.

Common Neighbors: Users who share a significant number of mutual connections, often used to find friends with similar networks. **User Behavior:** Analyzing user activities and lifestyle, beyond just direct connections, can further refine friend recommendations.

Social networks use GPS data and user behavior analysis for friend suggestions by identifying individuals with shared locations or interests, often creating "geo-fences" or using common check-in behaviors to find nearby people. **Social Network Friend Suggestions**

- **Location-based Matching:** Social networks use GPS data to find users in proximity, allowing for suggestions of people who are physically nearby or have visited common locations.
- **Behavioral & Lifestyle Analysis:** By analyzing user check-in behaviors and activities, systems can find individuals who share similar lifestyles and interests, leading to more relevant friend recommendations.

CS3301 DS

- **Social Graph Integration:** These features are combined with existing social graph data (user profiles, interests, shared friends) to build a more comprehensive understanding of users, improving the accuracy of friend suggestions.
- **Common Interests:** Systems can identify common interests by looking at user-generated content, tagged photos, or groups joined, then link these interests

to location-based suggestions for people with similar preferences. **3.8 GPS AND ROUTE FINDING**

Graphs and networked systems are fundamental to GPS and route finding by modeling road networks as graphs, with intersections as vertices and roads as edges.

Graph algorithms like the Dijkstra algorithm then find the shortest or most efficient paths by considering factors like one-way streets, turning restrictions, and speed limits.

This allows for crucial applications such as **emergency services routing, business logistics, and recreational travel planning**, ensuring reliable and optimal route calculations.

How Graphs Model Road Networks

Vertices (Nodes):

Intersections of roads are represented as vertices.

Edges (Links):

Roads connecting two intersections are represented as edges or links.

Directed/Weighted Edges:

Edges can be directed to represent **one-way streets** and weighted to include factors like **travel time, distance, or cost**.

Graph Representation

Adjacency List: Each node stores a list of its neighboring nodes and the edge weights.

Adjacency Matrix: 2D matrix to store weights between every pair of nodes (less efficient for sparse maps).

CS3301 DS

Algorithms for Route Finding

Shortest Path Algorithms:

Algorithms, such as the Dijkstra algorithm, are used to calculate the shortest path between two vertices (a starting point and a destination) within the graph model. **A* Algorithm**

An extension of Dijkstra's algorithm. Uses heuristics (like straight-line distance to destination) to guide the search. More efficient for large maps. **c.**

Bellman-Ford Algorithm

Handles graphs with negative weights (rare in GPS but used in some cases). **d. Floyd-Warshall Algorithm**

Finds shortest paths between every pair of nodes. Useful for precomputing routes in smaller maps.

How Route Finding Works in GPS Apps

1. Map Loading: Load the graph data representing the map.
2. User Input: Get start and destination points.
3. Node Matching: Match GPS coordinates to closest nodes on the graph.
4. Run Algorithm: Use Dijkstra or A* to find shortest/fastest path.
5. Return Route: Show the path as directions on the map.

Network Analysis:

Beyond just finding the shortest path, network analysis uses graph theory to construct isochrones (areas reachable within a certain time) and to perform resource allocation.

Applications in GPS and Navigation

Emergency Services:

Enables fast routing for ambulances and fire services to reach critical locations efficiently.

Logistics and Transportation:

Crucial for optimizing delivery routes for businesses, planning complex logistics, and guiding holidaymakers through a region.

CS3301 DS

Real-time Navigation:

Allows for the continuous recalculation of routes when new information is received, such as traffic changes or road closures.

Geographic Information Systems (GIS):

Network data is a core component of GIS, used for a wide range of operations that go beyond simple route finding.

GPS and Route Finding for Friend Discovery

- **Contextual Friend Suggestions:** GPS data provides context for social interactions; for instance, a user might discover a friend of a friend because they checked into the same event or location.

- **Shared Trajectories:** Analyzing user GPS trajectories can reveal common routes or patterns of movement, suggesting potential connections between users who travel similar paths or visit similar places.
- **Local Expertise & Popularity:** To find friends, applications might consider users who are local experts or frequent popular locations, using this information to enrich the context for recommendations.
- **Enhanced Social Experiences:** By integrating location and social features, platforms can make it easier for users to find and meet others with similar interests, whether online or in their local community.

Example (Simplified)

Imagine a small city with 4 intersections (A, B, C, D):

A connected to B (distance 5)

B connected to C (distance 3)

A connected to D (distance 10)

D connected to C (distance 1)

Graph edges:

$A \rightarrow B$ (5)

$B \rightarrow C$ (3)

$A \rightarrow D$ (10)

$D \rightarrow C$ (1)

Find shortest path from A to C:

CS3301 DS

$A \rightarrow B \rightarrow C = 5 + 3 = 8$

$A \rightarrow D \rightarrow C = 10 + 1 = 11$

Shortest path: $A \rightarrow B \rightarrow C$ with total distance 8.

3.9 DEPENDENCY RESOLUTION (COURSE PLANNING, SOFTWARE BUILDS)

Dependency resolution is a classic real-world application of graphs in data structures, especially in areas like:

- Course planning (prerequisites between courses),
- Software builds (compilation dependencies between files/modules),
- Package managers (installing libraries with dependencies).

What is

Dependency Resolution?

Dependency resolution is the process of determining the correct order in which

tasks, courses, or modules must be completed or built, such that all dependencies are satisfied before something is started.

This graph structure, a **dependency graph**, allows for the use of graph algorithms to find a valid order of operations, known as a topological sort. For course planning, it ensures all prerequisites are met before a student takes a subsequent course.

In software builds, it ensures that components are compiled and linked in the correct order, preventing errors caused by missing or unresolved dependencies.

How It Maps to Graphs

This is modeled using a Directed Graph (Digraph):

Nodes (Vertices): Tasks, courses, modules, files, etc.

Directed Edges: $A \rightarrow B$ means A must be completed before B (i.e., B depends on A).

This forms a Directed Acyclic Graph (DAG) in most cases.

Applications

a. Course Planning

Vertices: Courses

CS3301 DS

Edges: Directed from prerequisite to dependent course.

Problem: Find a valid order to take all courses.

Solution: Topological Sorting

b. Software Builds

Vertices: Source files/modules

Edges: $A \rightarrow B$ means A must be compiled before B.

Problem: Find correct build order.

Solution: Topological Sorting

c. Package Managers (e.g., npm, pip)

Packages depend on other packages.

Dependency tree = DAG.

Problem: Install in the right order and detect circular dependencies (cycles).

Solution: Use DFS or Kahn's algorithm for topological

sorting. **Topological Sorting (Core Algorithm)**

Used to linearize the nodes of a DAG such that for every edge $U \rightarrow V$, U comes before V in the order.

Algorithms:

DFS-based Topological Sort

Kahn's Algorithm (BFS-based)

Example: Course Planning

Suppose you have 4 courses:

Course 0 $\rightarrow$ no prerequisites

Course 1 $\rightarrow$ depends on 0

Course 2 $\rightarrow$ depends on 0

Course 3 $\rightarrow$ depends on 1 and 2

Graph:

$0 \rightarrow 1$

$0 \rightarrow 2$

$1 \rightarrow 3$

$2 \rightarrow 3$

CS3301 DS

Valid course order: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ or $0 \rightarrow 2 \rightarrow 1 \rightarrow 3$

Algorithm for topological ordering

1. Calculate in-degree (number of incoming edges) of all nodes. 2. Add nodes with in-degree = 0 to a queue.

3. While the queue is not empty:

- * Remove a node from the queue and add it to the topological order.

- * For each neighbor of the removed node:

- * Reduce its in-degree by 1.

- * If in-degree becomes 0, add it to the queue.

Cycle Detection

If the dependency graph has a cycle, it means there's a circular dependency. In course planning $\rightarrow$ can't finish all courses.

In builds $\rightarrow$ infinite loop / error.

Cycle detection is done using DFS or by checking if a full topological sort is not possible.